SQL Assignment - Medium

Now, to get a better understanding we will be extending the RetailDB. So, here we'll work with **5 tables**:

1. **Customers** – customer details
   (customer_id, name, email, city, signup_date**)**

2. **Orders** – order details
   (order_id, customer_id, order_date, total_amount, payment_mode**)**

3. **Products** – product details
   (product_id, product_name, category, price, stock_qty**)**

4. **Order_Items** – links orders with products
   (order_item_id, order_id, product_id, quantity, price_each**)**

5. **Suppliers** – product suppliers
   (supplier_id, supplier_name, contact_email, city**)**

After inserting some data, the Tables looks like:

Customers

customer_id	name	email	city	signup_date
1	Rohit Kumar	rohit.kumar@gmail.com	Delhi	2024-01-15
2	Sneha Sharma	sneha.sharma@yahoo.com	Mumbai	2024-02-20
3	Amit Patel	amit.patel@gmail.com	Ahmedabad	2024-03-05
4	Priya Reddy	priya.reddy@gmail.com	Hyderabad	2024-03-22
5	Karan Singh	karan.singh@outlook.com	Chennai	2024-04-10
6	Neha Verma	neha.verma@gmail.com	Pune	2024-05-08

Orders

order_id	customer_id	order_date	total_amount	payment_mode
1	1	2024-07-10	82498.00	UPI
2	2	2024-07-15	74999.00	Credit Card
3	3	2024-07-20	5498.00	Net Banking
4	4	2024-07-22	5998.00	Debit Card
5	5	2024-08-01	1999.00	UPI
6	6	2024-08-03	6499.00	Credit Card
7	1	2024-08-05	1499.00	UPI
8	2	2024-08-07	59999.00	Debit Card
9	3	2024-08-10	1497.00	UPI
10	4	2024-08-12	55999.00	Net Banking
11	5	2024-08-15	2999.00	Credit Card
12	6	2024-08-18	285.00	UPI
13	1	2024-08-20	499.00	Cash
14	2	2024-08-22	1999.00	UPI
15	3	2024-08-25	55999.00	Net Banking

Products

product_id	product_name	category	price	stock_qty	supplier_id
1	iPhone 15	Electronics	79999.00	25	1
2	Samsung Galaxy S24	Electronics	74999.00	30	2
3	Noise Smartwatch	Wearables	4999.00	100	2
4	Boat Earbuds	Wearables	2499.00	150	3
5	Kurta Set	Fashion	1999.00	200	3
6	Running Shoes	Fashion	2999.00	180	3
7	Prestige Mixer Grinder	Home Appliances	6499.00	75	4
8	Tata Tea 1kg Pack	Groceries	499.00	300	4
9	Amul Butter 500g	Groceries	285.00	400	4
10	Sony Bravia 55” TV	Electronics	59999.00	20	1
11	Lenovo Laptop	Electronics	55999.00	40	1
12	Philips Trimmer	Personal Care	1499.00	120	2

Order_Items

order_item_id	order_id	product_id	quantity	price_each
1	1	1	1	79999.00
2	1	4	1	2499.00
3	2	2	1	74999.00
4	3	3	1	4999.00
5	3	6	1	2999.00
6	4	5	3	1999.00
7	5	7	1	6499.00
8	6	12	1	1499.00
9	7	9	2	285.00
10	8	10	1	59999.00
11	9	8	3	499.00
12	10	11	1	55999.00
13	11	6	1	2999.00
14	12	9	1	285.00
15	13	8	1	499.00
16	14	5	1	1999.00
17	15	11	1	55999.00

Suppliers

supplier_id	supplier_name	contact_email	city
1	Reliance Digital	contact@reliancedigital.in	Mumbai
2	Croma Electronics	support@croma.com	Bengaluru
3	Big Bazaar Online	sales@bigbazaar.com	Delhi
4	DMart Suppliers	dmart@dmart.in	Pune

Instructions to Load the tables on MySQL Workbench:

1. *Download the files from folder link:*

https://drive.google.com/drive/folders/1fHOzGXkzKlOwmnLsiecWha4iQYlxf1qE?usp=drive_link

2. Create the database with name as - ***RetailDB_2***, with the following table as: (just copy paste on sql script)

```sql
-- 1. Customers
CREATE TABLE Customers (
    customer_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100),
    email VARCHAR(100),
    city VARCHAR(50),
    signup_date DATE
);

-- 2. Suppliers
CREATE TABLE Suppliers (
    supplier_id INT AUTO_INCREMENT PRIMARY KEY,
    supplier_name VARCHAR(100),
    contact_email VARCHAR(100),
    city VARCHAR(50)
);

-- 3. Products
```

```sql
CREATE TABLE Products (
    product_id INT AUTO_INCREMENT PRIMARY KEY,
    product_name VARCHAR(100),
    category VARCHAR(50),
    price DECIMAL(10,2),
    stock_qty INT,
    supplier_id INT,
    FOREIGN KEY (supplier_id) REFERENCES Suppliers(supplier_id)
);

-- 4. Orders
CREATE TABLE Orders (
    order_id INT AUTO_INCREMENT PRIMARY KEY,
    customer_id INT,
    order_date DATE,
    total_amount DECIMAL(10,2),
    payment_mode VARCHAR(50),
    FOREIGN KEY (customer_id) REFERENCES Customers(customer_id)
);

-- 5. Order_Items
CREATE TABLE Order_Items (
    order_item_id INT AUTO_INCREMENT PRIMARY KEY,
    order_id INT,
    product_id INT,
    quantity INT,
    price_each DECIMAL(10,2),
    FOREIGN KEY (order_id) REFERENCES Orders(order_id),
    FOREIGN KEY (product_id) REFERENCES Products(product_id)
);
```

3. *Load the files on the created tables in MySQL Workbench using fetch csv data.*

4. *Can view with select command.*

Task: Solve all the below mentioned problems by writing the SQL queries.

a) Normal Queries

1. Fetch all products along with their supplier name (INNER JOIN).

2. Find all customers and their orders, even if they have not placed any (LEFT JOIN).

3. Get all suppliers and the products they supply, even if no products exist for a supplier (RIGHT JOIN).

4. Show all customers and all orders (FULL OUTER JOIN simulation using UNION).

5. List all products priced between ₹5000 and ₹50,000 and supplied from "Mumbai".

b) Aggregations & Group By

6. Find the total number of orders placed by each customer and show only those who placed more than 2 (GROUP BY + HAVING).

7. Show each supplier's total sales value (sum of quantity × price_each).

8. Find the average, highest, and lowest price of products in each category.

9. Find the top 5 customers by total spending (ORDER BY SUM(total_amount) DESC LIMIT 5).

10. Show the number of unique products ordered by each customer.

c) Subqueries

11. Find customers who placed an order with an amount greater than the average order amount (subquery).

12. Find products that have never been ordered (subquery with NOT IN).

13. List customers who ordered at least one product from the "Electronics" category.

14. Get suppliers who provide products that have been ordered more than 100 times in total.

15. Find the most expensive product(s) using a subquery with MAX().

d) Advanced Filters

16. Show orders placed by customers who live in either Mumbai, Delhi, or Bengaluru (IN operator).

17. Show orders where payment mode is NOT UPI or Credit Card (NOT IN).

18. Find customers who have no email address recorded (IS NULL).

19. Show suppliers who are not from the same city as any customer (NOT IN subquery).

20. Get the latest 3 orders placed, skipping the first 2 (ORDER BY + LIMIT + OFFSET).